Topics for Final Exam

Tapti Palit

Design patterns

- Creational patterns: singleton, abstract factory
- Structural patterns: adapter, proxy (static proxy), decorator
- Behavioral: template method, state, observer, visitor
- Sample question topics:
  - When to use template method? (when specializing a part of an algorithm while reusing a generic skeleton)
  - Difference between proxy vs. decorator
  - Integrating different design patterns (e.g. a solution that requires observer pattern to notify multiple observers where each observer implements a variant of the same algorithm using the template method pattern)
- We won’t ask about programming language-level concepts (e.g. can you implement X design pattern if language feature Y was unavailable)

Reflection and annotations

- Reflection for introspection
- Bytecode generation for instrumentation and dynamic proxies
- Annotations for adding metadata to objects, methods
- Sample question topics
  - When to use static proxy vs dynamic proxies (static proxies work best if there's a single class you want to proxy; dynamic proxies work best when you want to apply the same proxying logic to multiple classes)
  - Annotations for applying proxy logic or reflection logic selectively to a field or method

Microservices

- Microservice pros and cons (scalability, low coupling; vs. higher latency, worse debuggability, etc)
- Data denormalization and resulting challenges
- Eventual consistency and resulting challenges
- Increased network overhead and resulting challenges
- Caching (KV stores eg. Redis/Memcached)
- DB storage engines
  - B+ trees vs. LSM trees
  - Tradeoffs (read vs. write-heavy workloads; sequential scans vs. random reads)
  - B+ trees might have to rebalance on writes -> slower writes, faster reads
  - LSM trees might have to scan all logs for random reads -> slower reads, faster writes
  - Sequential scans reasonably fast for LSM trees

Microservices (contd.)

- Communication styles
  - RPC: Text vs. binary (pros and cons: text-based is easily debuggable, binary is more efficient and type-safe by design)
  - Message-queues (push-based where the broker pushes the messages)
  - Kafka (pull-based where the clients pull events)
- Sample question topics
  - Design questions where you'd have to pick the best DB storage engine, choose between RPC, MQs, or Kafka, and decide whether a caching layer will be helpful or not
- We won't ask: Spring Boot, gRPC, JSON, MongoDB API (or any other API for that matter); Network layers; HTTP methods (GET, PUT, etc), Protobuf field encodings, SQL commands, etc.

Kafka

- Push-based vs. pull-based systems
- How is Kafka log different from LSM tree logs? (does not support random reads at all; stores events instead of records)
- Why is the partitioning key important? What happens if you choose the wrong partitioning key? (we lose ordering)
- Kafka consumer committed state and crash recovery
  - At-most once vs. at-least once delivery guarantees
- Kafka replication
- acks=0, acks=1, acks=all

Kafka (contd.)

- Sample question topics
  - Imagine we see a message being delivered twice after a Kafka consumer rebooted after a crash – how could this happen? (At-least once delivery guarantee)
  - If we want to tolerate N nodes going down, what should the replication factor be?
  - Use-cases where acks=0, acks=1, acks=all would be adequate
- We won't ask: any Kafka commands; details of Raft; metadata coordination

K8s

- Container vs. virtual machine
- What benefits do the pod abstraction provide over containers? (colocation)
- Label-based scheduling; pod affinity
- Fault tolerance and liveness probes
- Sample questions
  - Use cases for deploying chatty microservices on same pod, label-based scheduling
  - Use cases for picking the right liveness probe type
- We won’t ask: Docker commands, Dockerfile format, app descriptor format; exact configuration option names; etcd details; K8s networking details

Testing

- Blackbox testing -> greybox testing (coverage-guided) -> whitebox testing (symbolic execution)
- Input generators and mutators and when each approach is useful (byte-level mutators work for simple byte/string-based inputs; dictionaries give some structure; grammar-based mutators can generate fully valid inputs with complex grammar such as HTML, SQL, etc)
  - Random generation
  - Byte-level mutators
  - Dictionaries
  - Grammar-aware mutators
- Coverage types
  - Edge-based
  - Value-profile
  - Path-based
- Benefits and challenges of each mutation and coverage type (throughput vs. valid input, etc)

Testing (contd.)

- DGF (need for DGF, distance metric)
- Silent bugs and undefined behavior
- Sanitizers concepts
  - ASan implementation tradeoffs
- Symbolic execution benefits and challenges
- Sample questions
  - Which mutation strategy and coverage metric is best to find bugs in a particular app/program?
  - Do we need DGF for a particular use case?
  - Can symbolic execution find a particular bug in a piece of code – why or why not?
- We won't ask: details of buffer overflow, use-after-free, double-free, AFL, clang, gcc commands, KLEE API